

Styling Guide for Write (Stylus Labs)

This guide explains how to customize the visual appearance of the Write app's user interface — colors, corner radii, shadows, and interactive states (hover/pressed/checked/disabled). It does **not** cover document/canvas appearance (page background, ruling color, pen colors) — see §8 for that.

Note on links in this file: the Table of Contents below uses explicit HTML anchors (`<a id="sec-N">`) rather than auto-generated heading slugs, because GitHub-style slug generation (`#1-how-the-ui-is-styled`) is not part of the Markdown spec — every renderer implements it differently, and `glow/marktext` don't match GitHub's algorithm. See §12 if the links still don't jump in your viewer.

Table of Contents

1. [How the UI is styled — the big picture](#)
2. [The three pieces of `theme.cpp`](#)
3. [Changing colors](#)
4. [Changing corner radius](#)
5. [Changing shadows](#)
6. [Styling interactive states \(hover / pressed / checked / disabled / focused\)](#)
7. [Loading your own theme at runtime](#)
8. [That's UI styling — what about page background / pen colors?](#)
9. [Supported CSS properties reference](#)
10. [Recipes](#)
11. [Gotchas](#)
12. [Building & iterating quickly](#)

13. Why the ToC links may still not work in your viewer

1. How the UI is styled — the big picture

Write's GUI toolkit (`ugui`, a submodule of this repo) is **SVG-based**: every widget (button, menu, dialog, toolbar, textbox...) is literally an SVG fragment, and it's styled with an actual **CSS stylesheet** parsed by a CSS/SVG engine (`usvg`, also a submodule). If you've styled HTML with CSS before, most of this will feel familiar — selectors, classes, cascading, `var(--custom-properties)` — with a few desktop-app-specific properties bolted on (`box-shadow`, `border-radius`, layout properties like `flex-direction`).

There is no separate theming “system” or config UI beyond this — you edit CSS (and optionally the widget SVG markup) directly, either by editing the built-in defaults in the source, or by pointing the app at your own files at runtime (§7).

2. The three pieces of `theme.cpp`

Everything lives in `ugui/theme.cpp`, as three embedded C++ raw strings:

Variable	Purpose
<code>defaultColorsCSS</code>	The color palette — CSS custom properties (<code>--dark</code> , <code>--button</code> , <code>--checked</code> , etc.) for the dark theme and a <code>.light</code> override
<code>defaultStyleCSS</code>	The actual widget style rules (fills, opacity, shadows, corner radii, state classes) — consumes the palette via <code>var(--xxx)</code>
<code>defaultWidgetSVG</code>	The SVG markup prototypes for every widget type (button, checkbox, menu, dialog, slider, scrollbar, ...), referenced by CSS class name

These get parsed and applied at startup in `syncscribble/resources.cpp` (`createStylesheet()` and the code around it). You generally only need to touch `defaultColorsCSS`/`defaultStyleCSS`, or an external CSS file with equivalent rules (§7) — you rarely need to touch the SVG markup itself unless you’re adding/restructuring a widget’s shape.

3. Changing colors

3a. The palette (recommended entry point)

At the top of `theme.cpp`, `defaultColorsCSS` defines a palette as CSS custom properties on the root window node:

```
svg.window /* :root */
{
    --dark: #101010;      /* toolbar */
    --window: #303030;    /* menu, dialog */
    --light: #505050;     /* separators */
    --base: #202020;      /* list, inputbox */
    --button: #555555;
    --hovered: #32809C;
    --pressed: #32809C;
    --checked: #0000C0;
    --title: #2EA3CF;
    --text: #F2F2F2;
    --text-weak: #A0A0A0;
    --text-bg: #000000;
    --icon: #CDCDCD;
    --icon-disabled: #808080;
}

/* light theme */
svg.window.light
{
    --dark: #F0F0F0;
    --window: #DDDDDD;
    ...
}
```

Changing one of these variables re-colors every widget rule that consumes it via `var(--name)`, which is most of the stylesheet — e.g.:

```
.toolbar { fill: var(--dark); }
.pushbutton { fill: var(--button); }
.pushbutton.checked { fill: var(--checked); }
text { fill: var(--text); }
```

This is the easiest, lowest-risk way to reskin the app:
edit the hex values in defaultColorsCSS (or override them in your own stylesheet, see §7) and rebuild/reload.

Variable resolution works like real CSS custom properties: `var(--name)` is resolved by walking up the ancestor chain of the SVG tree looking for the nearest node that declares `--name`, so `svg.window.light` overriding `--dark` cascades down to every widget under it, exactly like `:root + .dark-mode` overrides in web CSS.

3b. One-off hardcoded colors

A handful of rules in defaultStyleCSS use literal hex colors instead of a variable, e.g.:

```
.toolbutton.checked .checkmark { fill: #0080FF; }
.tooltip { fill: #FFFFCF; }
.warning { fill: #FFFF00; }
.pushbutton.disabled { fill: #404040; }
```

Edit these directly, or better, promote them to a new `--` variable in the palette if you want the color reused elsewhere.

3c. color vs fill

Most rules use `fill` (this is SVG, not HTML — `fill` is what paints a shape). A few icon shapes are drawn with `stroke="currentColor"` in the SVG markup, in which case the CSS property that controls them is `color`, not `fill`:

```
.checkbox { color: var(--icon); }
.previewbtn { color: #808080; }
```

If changing `fill` on a rule doesn't seem to affect a stroked icon, try `color` instead (or check the widget's SVG in `defaultWidgetSVG` for `stroke="currentColor"`).

4. Changing corner radius

Write's default theme is **square-cornered** — no widget currently ships with rounded corners, but the mechanism is there:

Option A — CSS `border-radius` (recommended, no SVG editing needed). It works on `<rect>` elements only, and accepts the standard CSS 1-value or 4-value syntax (top-left, top-right, bottom-right, bottom-left):

```
.pushbtn-bg { border-radius: 8; }
.toolbutton .background { border-radius: 4; }
.inputbox-bg { border-radius: 6 6 6 6; }
```

Option B — SVG `rx/ry` attributes, set directly on a `<rect>` in `defaultWidgetSVG`. This is what the scrollbar handle already uses:

```
<rect id="scroll-handle" class="scroll-handle" box-
      anchor="vfill" width="4" height="20" rx="2"
      ry="2"/>
```

and the pushbutton's background even has a **commented-out example** ready to uncomment:

```
<rect class="background pushbtn-bg" box-anchor="hfill"
      width="36" height="36"/> <!-- rx="8" ry="8" --
>
```

Use whichever is more convenient: `border-radius` in CSS if you're only touching the stylesheet (e.g. via an external `guiCSS` file, §7); `rx/ry` if you're editing the widget SVG directly anyway.

5. Changing shadows

Menus and dialogs use CSS `box-shadow`, with standard syntax (`offset-x offset-y [blur] [spread] [inset] color, rgba() supported`):

```
.menu, .dialog { box-shadow: 0px 0px 10px 0px
                          rgba(0,0,0,0.5); }
```

Alternate examples are left commented out in `theme.cpp` for reference:

```

/* .menu, .dialog { box-shadow: 0px 0px 40px 0px
    rgba(0,0,0,0.40); } */ /* android-like */
/* .menu, .dialog { box-shadow: 6px 6px 4px -4px
    rgba(0,0,0,0.375); } */ /* offset, windows-like
*/

```

Just edit the values or swap in one of the alternates.
Shadows can be applied to any widget class, not just
.menu/.dialog.

6. Styling interactive states (hover / pressed / checked / disabled / focused)

Write does **not** use real CSS pseudo-classes like :hover or :checked. Instead, the app toggles plain CSS **classes** on a widget's SVG node at runtime (in C++, via `SvgNode::addClass()/removeClass()`), and the stylesheet targets those classes with ordinary compound selectors. So “the styling when a button is toggled” is just:

```

.pushButton.checked { fill: var(--checked); }
.toolbar.checked .checkmark { fill: #0080FF; }

```

The classes the app manages for you, and what triggers them:

Class	Applied when	Typical selector in CSS
.hovered	pointer enters the widget (no button held)	.toolbar.hovered
.pressed	actively pressed/ clicked	.pushbutton.pressed
.checked	a toggle button (checkbox, radio, sticky toolbar) is ON	.pushbutton.checked
.checked.once	a <i>non-sticky</i> checked toolbar (auto-releases)	.toolbar.checked.once .checkmark
.disabled	widget is disabled (setEnabled(false))	.pushbutton.disabled, .disabled .icon
.focused	text input has keyboard focus	.inputbox.focused .inputbox-bg
.light		svg.window.light

Class	Applied when	Typical selector in CSS
	on the window root only — light theme active	

To style a toggled/checked button, target `.<widget-class>.checked` (and optionally a descendant like `.checkmark` if you only want the check indicator, not the whole button, to change):

```
/* whole button re-colors when checked */
.pushbutton.checked { fill: #2E8B57; }

/* only tint the toolbutton's bottom "checkmark" bar,
   leave background alone */
.toolbutton.checked .checkmark { fill: #FF8800; }
```

You can combine states too, since these are just compound class selectors:

```
.pushbutton.checked.pressed { fill: #1B5E3A; }
```

Note existing quirks worth knowing: -

```
.checkbox .checkmark { display:
none; } / .checkbox.checked .checkmark { display:
block; } — checkboxes use display toggling (show/hide a
checkmark glyph) rather than a fill/color change. If you
want a checkbox's checkmark to also change color when
checked, add a .checkbox.checked .checkmark
{ fill: ...; } rule. - .disabled .icon (descendant
selector, not .icon.disabled) — the disabled class is set on
an ancestor container, not the icon itself, in some widgets.
If a new rule you write doesn't seem to apply, check
whether the state class lands on the widget itself or a
parent/child by searching theme.cpp for a similar existing
rule as a template.
```

7. Loading your own theme at runtime

You don't have to edit `theme.cpp` and recompile. Two config keys let you supply your own CSS and/or widget SVG at runtime:

Config key	Replaces	Default
<code>guiCSS</code>		"" (use built-in)

Config key	Replaces	Default
	defaultColorsCSS + defaultStyleCSS	
guiSVG	defaultWidgetSVG	"" (use built-in)

They're read in `syncscribble/resources.cpp` and each accepts **either**: - a literal multi-line CSS/SVG string (if the config value contains a newline), **or** - a **file path** (if the value is a single line — it's loaded via `readFile()`).

Set them by hand-editing the app's config file (they are **not** currently exposed in the in-app Preferences dialog):

- Linux: `~/ .config/styluslabs/write.xml`
- Android: `/sdcard/styluslabs/write.xml`
- iOS: `<Library>/write.xml`
- Windows: `write.xml` next to the executable / in the app-data dir

Add (or edit) the relevant entries, e.g.:

```
<guiCSS>/home/yourname/my-write-theme.css</guiCSS>
```

Important: if `guiCSS` is non-empty, the built-in `defaultColorsCSS` + `defaultStyleCSS` are **not** loaded at all — there's no merging/fallback. Your custom file must be a **complete** stylesheet (every rule the app needs), not just the overrides you want to change. The easiest way to make one: copy the contents of `defaultColorsCSS` + `defaultStyleCSS` out of `theme.cpp` into your own `.css` file, edit what you want, and point `guiCSS` at it. Same rule applies to `guiSVG` vs. `defaultWidgetSVG`.

For a quick **light/dark toggle only** (no custom colors), you don't need any of this — use the existing `uiTheme` Preferences option (Dark/Light), which just switches the `.light` class on the window root and is already wired into Preferences UI.

8. That's UI styling — what about page background / pen colors?

Out of scope for `theme.cpp/CSS`. Page background, ruling line color, and bookmark/link colors are **document/config values**, unrelated to the GUI theme system:

```
// syncscribble/scribbleconfig.cpp
cfg["pageColor"] = Color(Color::WHITE).argb();
cfg["ruleColor"] = Color(0, 0, 0xFF, 0x9F).argb();
cfg["bookmarkColor"] = Color(Color::BLUE).argb();
cfg["linkColor"] = Color(Color::BLUE).argb();
```

These are editable per-document in-app via the **Ruling dialog**, not via theme CSS. Selection-highlight color (blue/yellow outline when you select strokes) is computed automatically from page background luminance in `selection.cpp` and is not currently themeable at all.

9. Supported CSS properties reference

Standard SVG/CSS properties (parsed in `usvg/svgstyleparser.cpp`):

color, comp-op, display, fill, fill-rule, fill-opacity,
font-family, font-size, font-style, font-variant, font-weight,
offset, opacity, shape-rendering, stop-color, stop-opacity,
stroke, stroke-dasharray, stroke-dashoffset, stroke-linecap,
stroke-linejoin, stroke-alignment, stroke-miterlimit, stroke-opacity,
stroke-width, text-anchor, vector-effect, visibility, letter-spacing

App-specific extensions, layered on top by `ugui` (parsed in `ugui/svggui.cpp`, `Widget::updateLayoutVars()`), which flow through the same CSS cascade/selectors/`var()` machinery:

box-shadow, border-radius,
left, top, right, bottom, box-anchor, layout,
flex-direction, flex-wrap, justify-content, flex-break, margin

CSS custom properties (`--name: value`, consumed via `var(--name)`) are supported generally, not just for the built-in palette — you can define your own on any node and reference them anywhere below it in the tree.

There are **no real CSS pseudo-classes**

(:hover, :checked, :focus) — use the toggled classes from §6 instead (.hovered, .checked, .focused).

10. Recipes

Make the whole app use a teal accent instead of blue:

```
svg.window {  
  --checked: #00897B;  
  --hovered: #26A69A;  
  --pressed: #26A69A;  
  --title: #00897B;  
}
```

Round every button and input box:

```
.pushbtn-bg, .inputbox-bg { border-radius: 6; }
```

Make checked toolbar buttons pill-shaped and orange:

```
.toolbarbutton.checked .background { border-radius: 18;  
  fill: #FF9800; }
```

(You'll likely also want border-radius on the un-checked .background rect for consistent shape — or set it as a base .toolbarbutton .background rule so it doesn't change shape when toggled, only color.)

Flutter shadows:

```
.menu, .dialog { box-shadow: 0px 2px 4px 0px  
  rgba(0,0,0,0.25); }
```

11. Gotchas

- border-radius only rounds <rect> nodes — it has no effect on <circle>, <path>, etc. (irrelevant for those shapes anyway).
- guiCSS/guiSVG fully **replace** the defaults; there's no partial override/merge at the config-file level. Start from a full copy of theme.cpp's CSS if you go this route.
- guiCSS/guiSVG aren't in the Preferences UI — you must hand-edit write.xml, and restart the app to pick up changes.
- Selectors are plain classes/tags, not real pseudo-classes — search theme.cpp for an existing similar rule before guessing a selector for a new state, since which node

gets which class (self vs. ancestor vs. descendant)
varies per widget.

- Colors defined with `fill` won't affect
`stroke="currentColor"` shapes — use `color` for those
(see §3c).

12. Building & iterating quickly

There is no hot-reload / live-reload of the theme.

`defaultColorsCSS`, `defaultStyleCSS`, and `defaultWidgetSVG` are parsed exactly once, at startup (`setupResources()`/
`createStyleSheet()` in `syncscribble/resources.cpp`, called once from `application.cpp`'s init path). There's no file-watcher and no in-app “reload theme” command — every change requires restarting the process. (If you're driving the theme via an external `guiCSS/guiSVG` file per §7, that file is also only read at startup — editing it doesn't require a *recompile*, but still requires a *restart*.)

Fastest edit→see-it-running loop when editing `ugui/theme.cpp` directly:

`theme.cpp` isn't compiled as its own translation unit — it's textually `#included` by `syncscribble/resources.cpp` (`#include "ugui/theme.cpp"`). That means it's a dependency of exactly one object file, `resources.o`, and nothing else rebuilds when you touch it:

```
cd syncscribble
make USE_SYSTEM_SDL=1 DEBUG=1
```

What happens: only `resources.cpp` recompiles (one file, -O0 under `DEBUG=1` so it's quick), then the full binary relinks (a few seconds, independent of how much source you have). No other `.cpp` files in `ugui/syncscribble/` get touched. `DEBUG=1` builds land in `Debug/` instead of `Release/` and use -O0 (much faster to compile than the -O2 Release default) — use it for this kind of iteration, drop it for a final build. Then just run `./Debug/Write` (copy `scribblers/fonts` into `Debug/` first if you haven't already, same as the Release setup).

If you'd rather avoid any recompiling at all while tweaking colors: put your working copy of the CSS in an external file and point `guiCSS` at it in `write.xml` (§7) — then each iteration is just “edit file, restart app”, no `make` involved.

This is usually the faster loop for pure color/radius/shadow tweaks; drop the result back into `theme.cpp` once you're happy with it.

13. Why the ToC links may still not work in your viewer

Markdown itself never standardized “click a heading in a Table of Contents and jump to it” — that's a convenience each renderer bolts on by turning heading text into an `#anchor` slug, and every renderer's slugging algorithm is slightly different (case-folding, punctuation stripping, em-dash/space handling, etc.). GitHub, `marktext`, and `glow` don't agree with each other.

This file sidesteps the slug-matching problem by using **explicit HTML anchors** — `<a id="sec-N"></a>` placed right above each heading — with the ToC linking to those fixed ids (`#sec-1`, `#sec-2`, ...) instead of relying on auto-generated slugs. Raw inline HTML is part of the CommonMark spec, so any spec-compliant renderer should pass it through and honor the `id`. This should now work correctly in `marktext`, which previews Markdown as real HTML in an Electron/webview context.

`glow` is a different case and can't be fixed from the Markdown side. `glow` renders into a terminal pager (via `glamour`), not a browser/webview — there is no HTML DOM, no `id` attributes, and no concept of scrolling to a fragment. Terminal output is just text; `[label](#anchor)` links render as inert (or, if your terminal supports OSC-8 hyperlinks, clickable-but- pointing-nowhere-useful) text, and there is currently no way to make same-document anchor jumps work in `glow`, regardless of how the anchors are authored. If you need in-terminal navigation, `glow`'s own pager search (`/` to search, in the interactive pager) is the practical substitute for a clickable ToC.